

DAY 5 — BACKEND TASKS

TASK 1 — CREATE PRODUCT API STRUCTURE

Aim

The objective of this task is to create the basic backend structure for a Product API using:

- Node.js
- Express.js
- MongoDB
- Mongoose

This backend application will be used to store and manage product details.

Step 1 — Create Project Folder

Create a new folder named:

`product-api`

Create the following folders:

- models
- routes
- controllers
- config

Create these files:

- server.js
- .env
- package.json

Initialize the Node.js project and install:

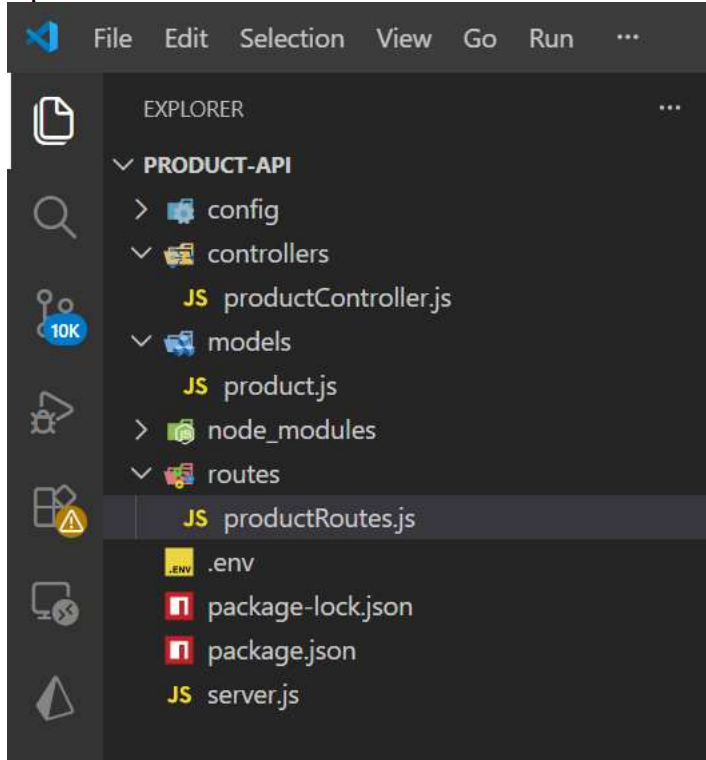
- express
- mongoose
- dotenv
- nodemon

Setup:

DAY 5 — BACKEND TASKS

- Create a basic Express server
- Connect MongoDB

Open the folder in Visual Studio Code.



Step 2 — Initialize Node.js Project

Open terminal in VS Code and run:

```
npm init -y
```

This command creates a `package.json` file automatically.

The `package.json` file stores:

- project name
- dependencies
- scripts
- version details

Step 3 — Install Required Packages

DAY 5 — BACKEND TASKS

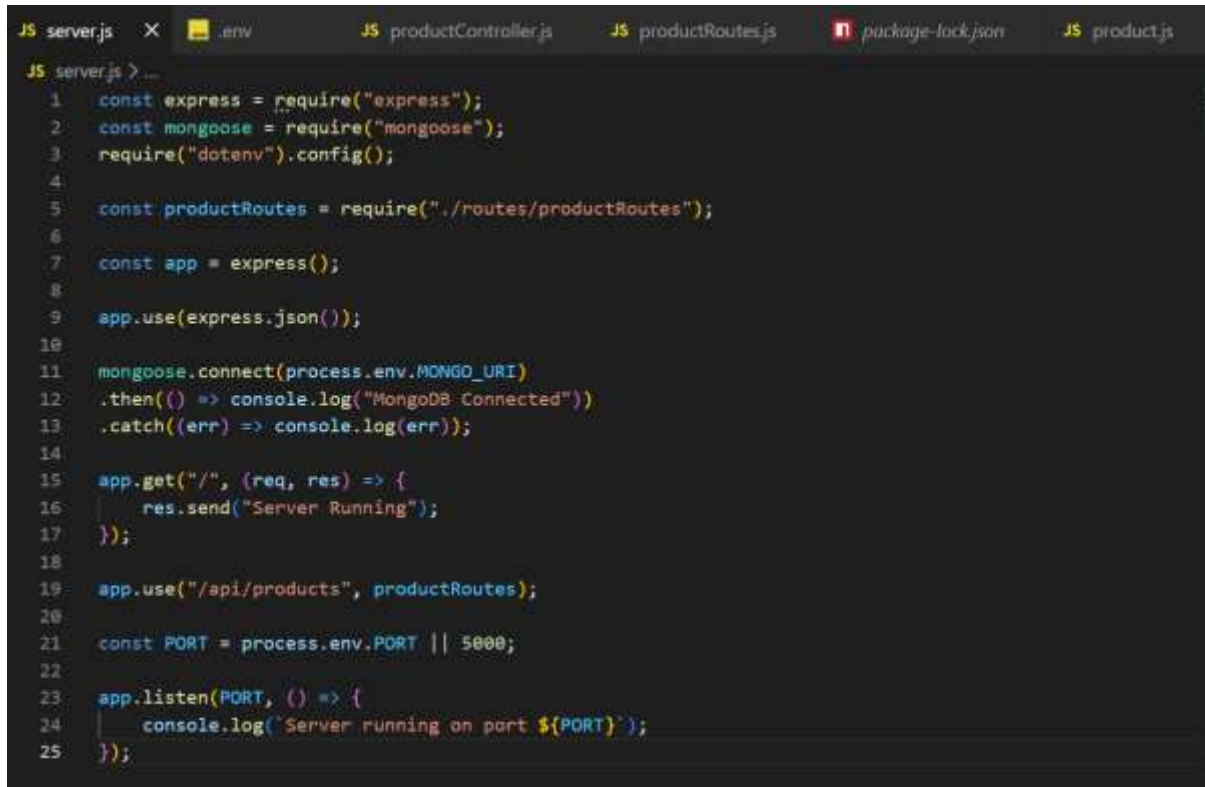
Install the required backend packages using:

```
npm install express mongoose dotenv
```

Install nodemon as development dependency:

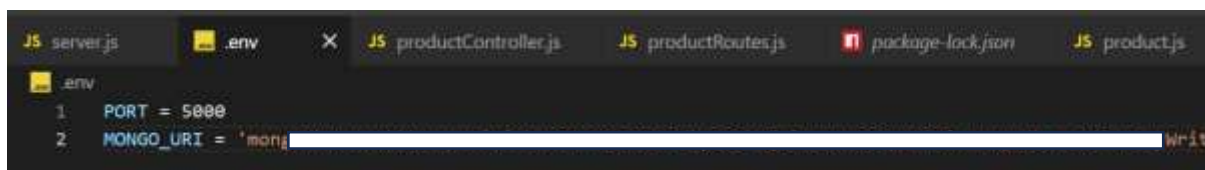
```
npm install nodemon --save-dev
```

Server.js



```
JS server.js X .env JS productController.js JS productRoutes.js package-lock.json JS product.js
JS server.js > ...
1  const express = require("express");
2  const mongoose = require("mongoose");
3  require("dotenv").config();
4
5  const productRoutes = require("./routes/productRoutes");
6
7  const app = express();
8
9  app.use(express.json());
10
11 mongoose.connect(process.env.MONGO_URI)
12 .then(() => console.log("MongoDB Connected"))
13 .catch((err) => console.log(err));
14
15 app.get("/", (req, res) => {
16   res.send("Server Running");
17 });
18
19 app.use("/api/products", productRoutes);
20
21 const PORT = process.env.PORT || 5000;
22
23 app.listen(PORT, () => {
24   console.log(`Server running on port ${PORT}`);
25 });
```

.env



```
JS server.js .env X JS productController.js JS productRoutes.js package-lock.json JS product.js
.env
1  PORT = 5000
2  MONGO_URI = 'mongi' Writ
```

DAY 5 — BACKEND TASKS

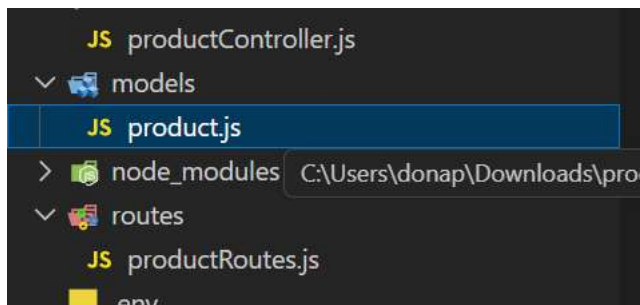
TASK 2 — Build Product Details API

Create a Product model with the following fields:

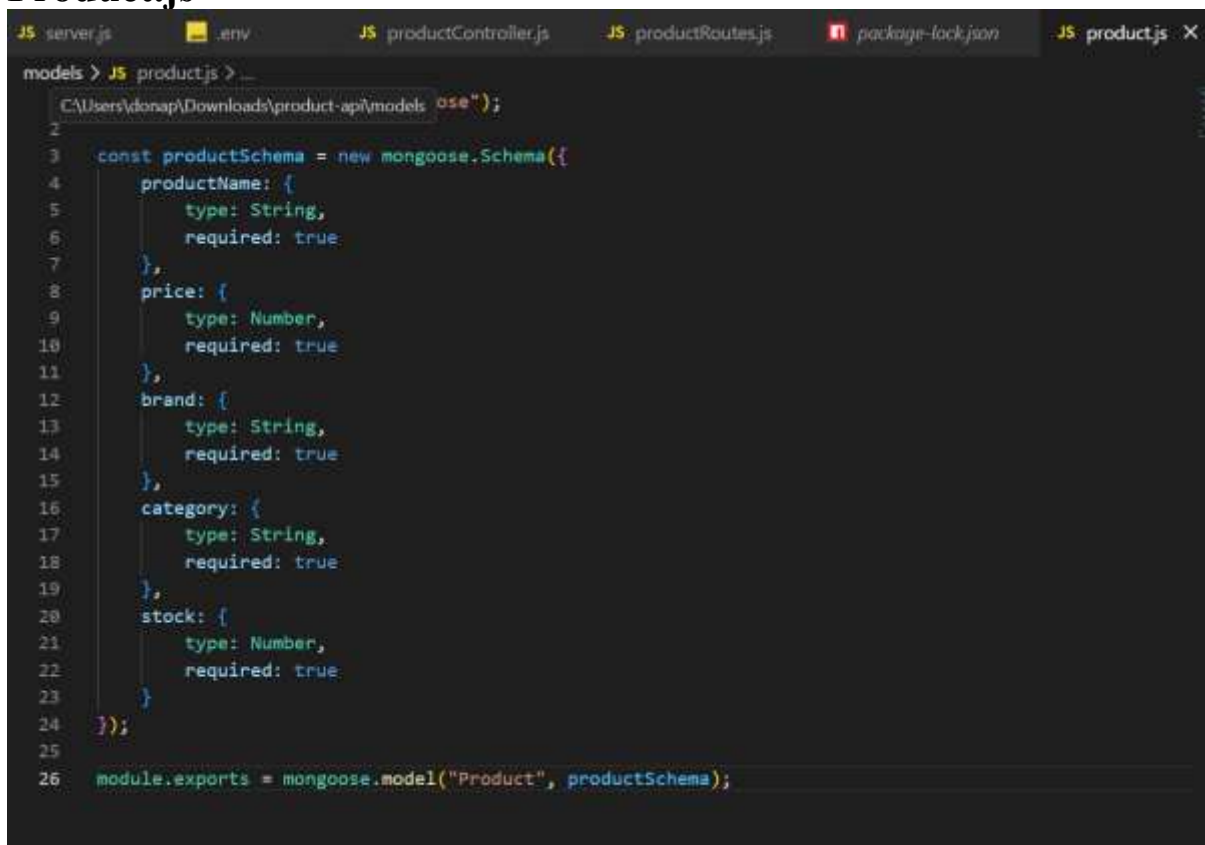
- productName
- price
- brand
- category
- stock

Create:

- Product model

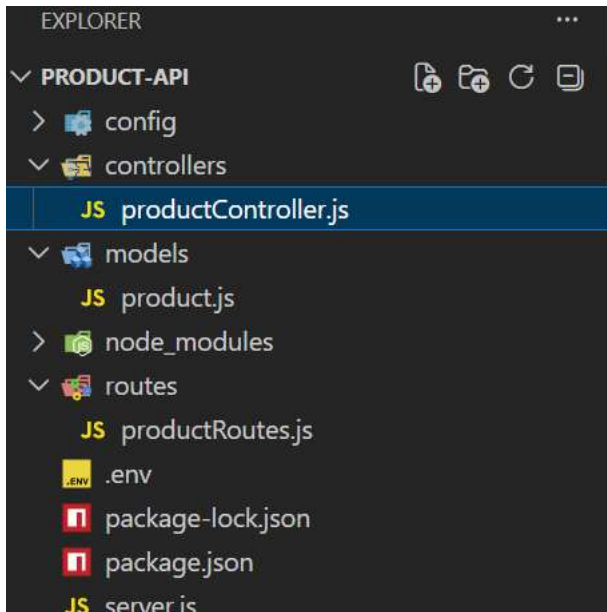


Product.js



DAY 5 — BACKEND TASKS

- Product controller

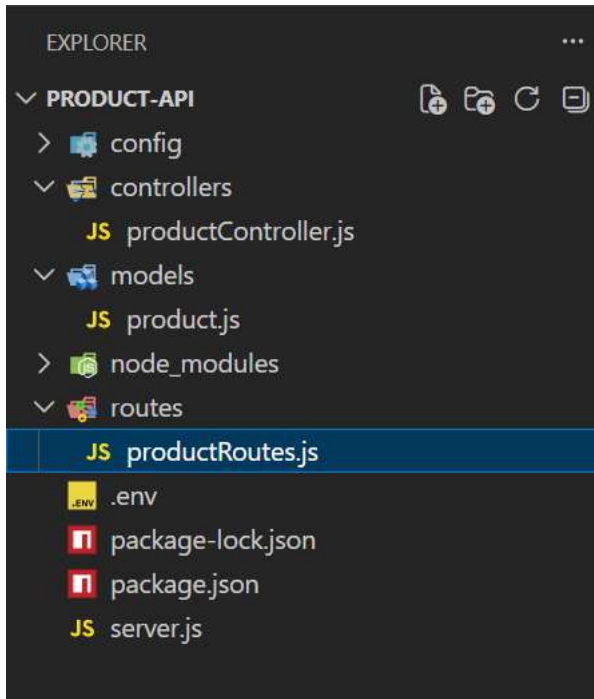


ProductController.js

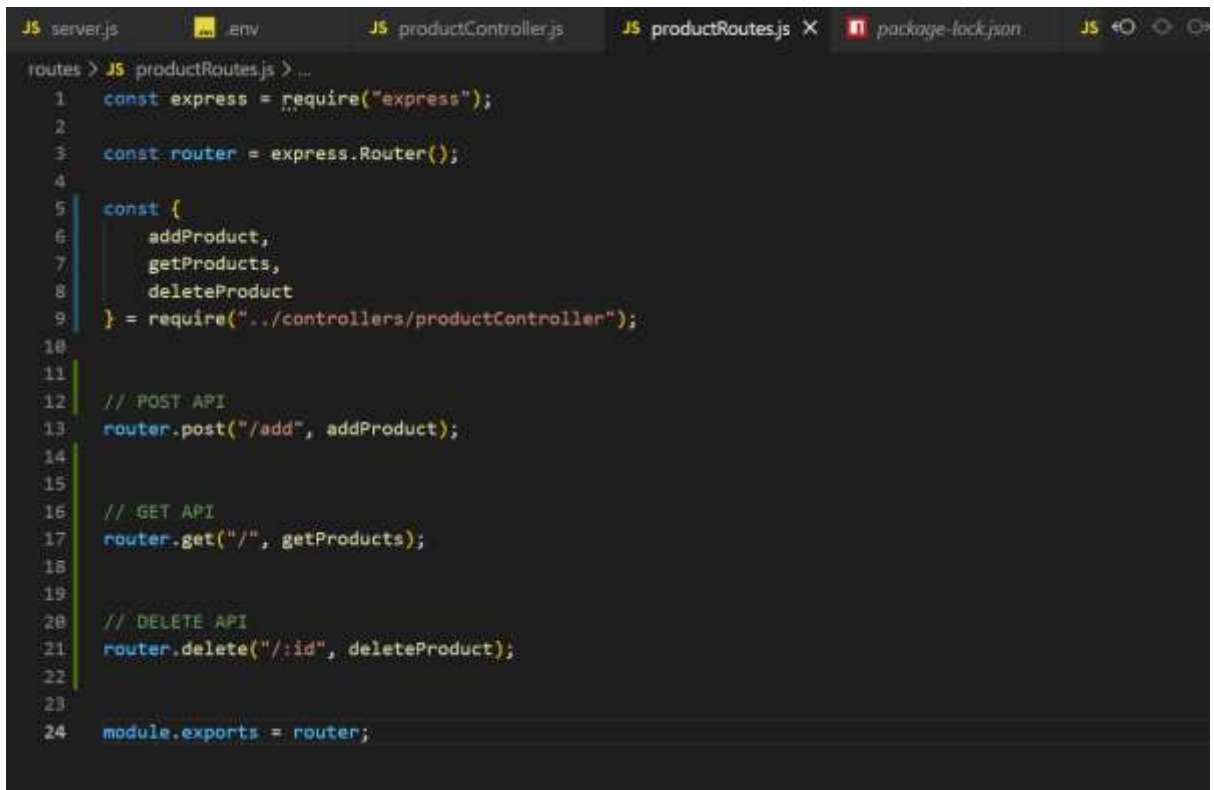
```
1 // Import dependencies
2 const Product = require("../models/Product");
3
4 // Import http module
5 const http = require("http");
6
7 // Import express module
8 const express = require("express");
9
10 // Import body-parser module
11 const bodyParser = require("body-parser");
12
13 // Import mongoose module
14 const mongoose = require("mongoose");
15
16 // Import dotenv module
17 const dotenv = require("dotenv");
18
19 // Load environment variables from .env file
20 dotenv.config();
21
22 // Create an instance of express
23 const app = express();
24
25 // Use body-parser middleware
26 app.use(bodyParser.json());
27
28 // Use mongoose
29 mongoose.connect(process.env.MONGO_URI, {
30   useNewUrlParser: true,
31   useUnifiedTopology: true,
32 });
33
34 // Create a REST API
35 // GET API
36 // Get all products
37 const getAllProducts = async (req, res) => {
38   try {
39     const products = await Product.find();
40     res.status(200).json(products);
41   } catch (error) {
42     res.status(500).json({
43       message: error.message,
44     });
45   }
46 }
47
48 // POST API
49 // Add a new product
50 const addProduct = async (req, res) => {
51   try {
52     const product = new Product(req.body);
53     await product.save();
54     res.status(201).json({
55       message: "Product Added successfully",
56       product,
57     });
58   } catch (error) {
59     res.status(500).json({
60       message: error.message,
61     });
62   }
63 }
64
65 // DELETE API
66 // Delete a product
67 const deleteProduct = async (req, res) => {
68   try {
69     const deleteProduct = await Product.findByIdAndDelete(req.params.id);
70     if (!deleteProduct) {
71       return res.status(404).json({
72         message: "Product Not Found",
73       });
74     }
75     res.status(200).json({
76       message: "Product Deleted successfully",
77     });
78   } catch (error) {
79     res.status(500).json({
80       message: error.message,
81     });
82   }
83 }
84
85 // Start the server
86 const server = http.createServer(app);
87 server.listen(process.env.PORT, () => {
88   console.log(`Server is running on port ${process.env.PORT}`);
89 });
```

DAY 5 — BACKEND TASKS

- Product routes



productRoutes.js



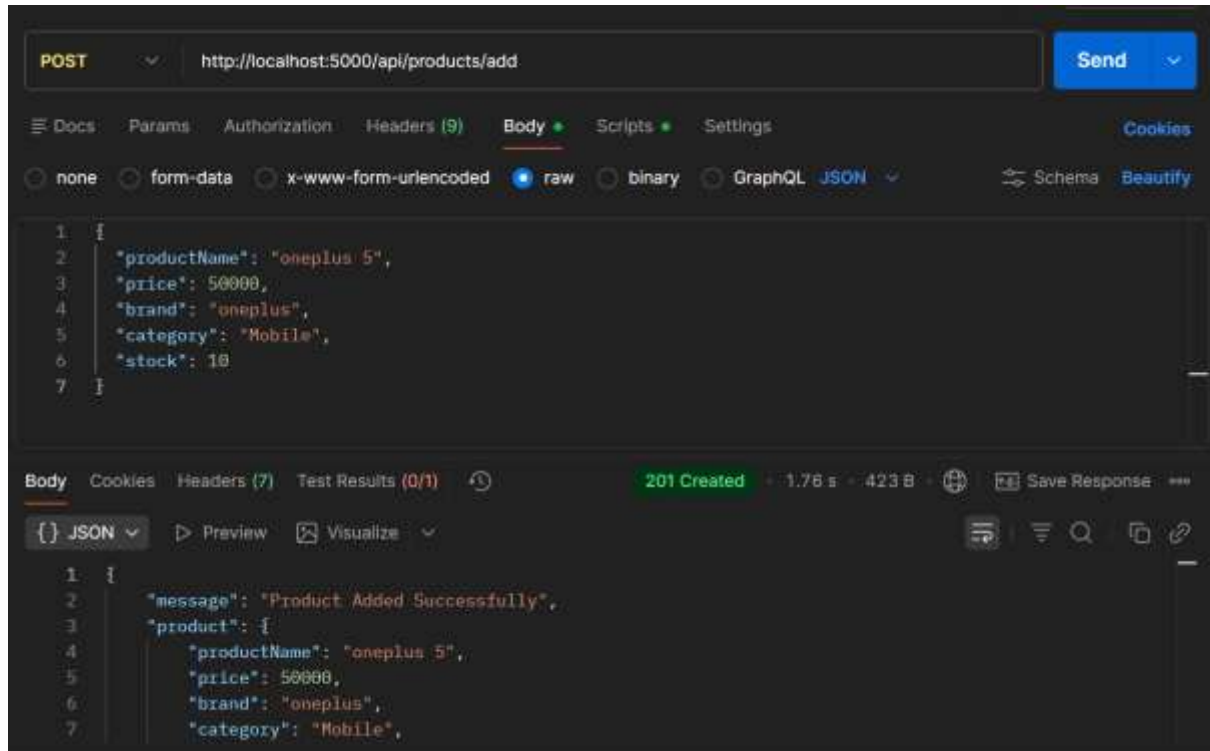
DAY 5 — BACKEND TASKS

Create POST API

API Endpoint:

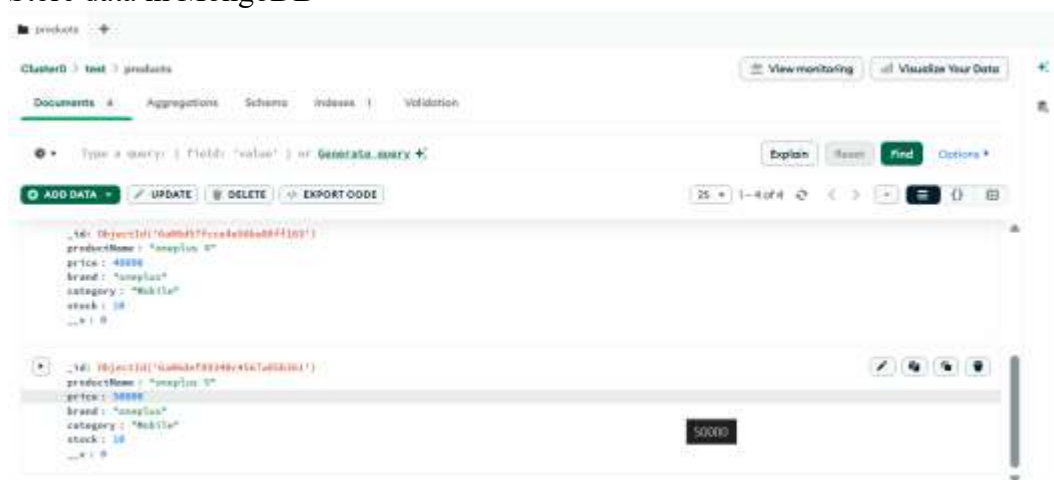
/api/products/add

In Postman



The API should:

- Accept product data
- Store data in MongoDB



-
- Return success message

DAY 5 — BACKEND TASKS

TASK 3 — Create Get Products API

Create GET API

API Endpoint:

/api/products

The API should:

- Fetch all products from MongoDB
- Return product list
- **In postman**

